

EMPIRICAL EVALUATION OF DEEP LEARNING INFERENCE ON ESP32-S3 MICROCONTROLLER

Jahangir Alipournajmi

Department of Artificial Intelligence Engineering
Hacettepe University
jahangir26@hacettepe.edu.tr

Hakan Yörük

Department of Artificial Intelligence Engineering
Hacettepe University
hakanyoruk26@hacettepe.edu.tr

ABSTRACT

The abstract paragraph should be indented 1/2 inch (3 picas) on both left and right-hand margins. Use 10 point type, with a vertical spacing of 11 points. The word ABSTRACT must be centered, in small caps, and in point size 12. Two line spaces precede the abstract. The abstract must be limited to one paragraph.

1 INTRODUCTION

The rapid growth of Internet of Things (IoT) devices has significantly increased the demand for intelligent data processing on resource-constrained hardware. Traditional cloud-based machine learning approaches provide substantial computational capabilities; however, they introduce critical limitations such as increased latency, high bandwidth consumption, continuous network dependency, and potential privacy risks due to data transmission to remote servers. These limitations make cloud-centric solutions less suitable for real-time, distributed, energy-constrained, and privacy-sensitive applications.

Tiny Machine Learning (TinyML) has emerged as a promising solution by enabling machine learning models to run directly on microcontroller-based devices. By performing inference locally on embedded systems, TinyML reduces dependency on cloud infrastructure and enables low-latency, energy-efficient, and privacy-preserving data processing (Tsoukas et al., 2024; Heydari & Mahmoud, 2025). This paradigm shift allows intelligent decision-making to occur closer to the data source, which is essential for time-sensitive embedded applications.

Recent advances in TinyML have accelerated its adoption in Industrial Internet of Things (IIoT) environments, where real-time processing, energy efficiency, and system reliability are critical requirements. TinyML has been investigated in various industrial use cases, including predictive maintenance, anomaly detection, condition monitoring, and process optimization, demonstrating its potential as an alternative to traditional cloud-based IoT systems (Alharthi et al., 2026). These developments highlight the growing importance of TinyML in bridging the gap between resource-constrained embedded devices and intelligent edge computing systems.

Despite these advantages, deploying deep learning models on microcontrollers remains challenging due to strict hardware constraints, including limited memory capacity, low computational power, and energy limitations. Vision-based TinyML tasks further increase these challenges because image classification and object detection models typically require higher computational and memory resources than many sensor-based applications. To address these constraints, model optimization techniques such as quantization, pruning, compression, and architecture-level simplification are widely used to reduce model size and computational complexity while maintaining acceptable accuracy. (Tsoukas et al., 2024; El Zeinaty et al., 2025).

ESP32-based microcontrollers have gained attention for low-cost edge AI applications due to their accessibility, wireless connectivity, and suitability for embedded IoT systems. Recent studies have shown that ESP32-based platforms can support embedded vision and edge computing tasks, including hybrid edge/cloud approaches that reduce the computational burden on microcontroller-class devices (Chang et al., 2025). In addition, ESP32-CAM-based studies have demonstrated the feasibility of CNN-based image classification on ultra-low-cost microcontroller hardware (Ben Aicha, 2025). However, the evaluation of lightweight vision models on the ESP32-S3 platform, particularly

in terms of quantization, inference latency, model size, and memory usage, remains an important research direction.

In this context, this study focuses on the implementation and evaluation of lightweight vision models on the ESP32-S3 microcontroller platform using pre-collected datasets rather than live camera streaming. Specifically, a MobileNet-based model is evaluated for image classification, while a YOLO-based model is evaluated for object detection under resource-constrained embedded conditions. The proposed approach aims to analyze the trade-offs between accuracy, inference latency, model size, and memory usage in embedded environments. The findings of this study are expected to provide a foundation for future real-time TinyML deployments on ESP32-S3 platforms. As future work, the dataset-based evaluation will be extended to real camera images for live image classification and object detection scenarios.

2 RELATED WORKS

Tiny Machine Learning (TinyML) has accelerated research on deploying machine learning and deep learning models on resource-constrained edge devices. By enabling inference directly on microcontroller-class hardware, TinyML reduces dependence on cloud infrastructure and supports low-latency, energy-efficient, and privacy-preserving data processing. Existing surveys emphasize that successful TinyML deployment requires more than compact model design; hardware limitations, embedded software frameworks, memory management, and energy constraints must also be considered together. Therefore, TinyML systems require a hardware–software co-design perspective in which model optimization and hardware-aware deployment are addressed jointly (Tsoukas et al., 2024; Heydari & Mahmoud, 2025).

TinyML has also become increasingly relevant in Industrial Internet of Things (IIoT) applications, where real-time operation, reliability, and energy efficiency are critical requirements. In industrial scenarios, TinyML-based approaches have been investigated for predictive maintenance, anomaly detection, condition monitoring, and process optimization. These studies indicate that TinyML can provide a practical alternative when cloud-based inference is limited by connectivity requirements, data transmission costs, or privacy constraints (Alharthi et al., 2026). Nevertheless, deploying machine learning models on low-power embedded devices remains technically challenging due to limited memory, restricted computational capacity, and strict energy budgets (Tsoukas et al., 2024; Heydari & Mahmoud, 2025).

To address these constraints, optimization techniques such as quantization, pruning, model compression, and hardware-aware model design are widely used in TinyML research. Quantization reduces the numerical precision of model parameters and operations to decrease model size and computational cost, while pruning and compression techniques reduce redundant parameters to support deployment on constrained hardware. These optimizations are particularly critical in embedded vision applications, since image-based models generally require more memory and computational resources than many sensor-based TinyML tasks.

Vision-based TinyML studies are commonly structured around two main tasks: image classification and object detection. For image classification, MobileNet-based lightweight convolutional neural network architectures are suitable candidates for embedded deployment due to their low parameter count and efficient computational structure. For object detection, compact variants of the YOLO family are widely adopted because they provide a practical balance between detection accuracy and inference speed. However, object detection is more computationally demanding than image classification, as the model must predict not only the object class but also its location. Therefore, deploying MobileNet-based classification models and YOLO-based object detection models on microcontrollers requires careful analysis of model optimization, input resolution, quantization effects, inference latency, and memory consumption (El Zeinaty et al., 2025).

ESP32-based microcontrollers have attracted attention for embedded AI and edge computing studies due to their low cost, wireless connectivity, and suitability for IoT-oriented applications. Previous ESP32-based edge computing studies show that these platforms can be used for embedded vision and object detection tasks. In addition, hybrid edge/cloud architectures have been proposed, where part of the processing is performed on the edge device while more computationally intensive operations are supported by external resources. Although such approaches can reduce the computational burden

on the microcontroller, they may also introduce additional limitations such as network dependency, communication latency, and increased system complexity. Therefore, evaluating which visual AI tasks can be executed directly on the microcontroller is important for understanding the practical limits of fully embedded TinyML deployment (Chang et al., 2025).

Within the ESP32 family, ESP32-CAM has become an important research platform for embedded vision applications because it combines a low-cost microcontroller with an integrated camera module. Ben Aicha (2025) developed an optimized embedded AI system for real-time image classification on ESP32-CAM using a quantized CNN model. This study demonstrated the feasibility of CNN-based image classification on ultra-low-cost hardware and highlighted system-level engineering challenges such as memory limitations, TensorFlow Lite deployment issues, task scheduling, and the coordination of AI inference with camera and communication modules. These findings indicate that embedded AI performance is not determined solely by the neural network architecture, but is also strongly affected by memory management, runtime stability, preprocessing, and hardware-aware implementation decisions (Ben Aicha, 2025).

Overall, the existing literature demonstrates the feasibility of TinyML and ESP32-based embedded vision applications. However, many studies focus on general TinyML principles, industrial IoT applications, hybrid edge/cloud architectures, or specific ESP32-CAM-based scenarios. In contrast, the dataset-based evaluation of MobileNet-based image classification and YOLO-based object detection models on the ESP32-S3 platform remains limited, particularly with respect to quantization, inference latency, model size, and memory usage. ESP32-S3 is a relevant low-cost microcontroller platform for embedded IoT and edge AI experimentation, but practical deployment of visual models on this platform requires detailed analysis of performance and resource utilization.

Based on this gap, this study evaluates lightweight vision models on the ESP32-S3 microcontroller using pre-collected datasets instead of live camera streaming. Specifically, a MobileNet-based model is examined for image classification, while a YOLO-based model is evaluated for object detection. Unlike previous studies that primarily focus on real-time camera-based deployment or hybrid edge/cloud execution, this work adopts a controlled dataset-driven evaluation approach to analyze the trade-offs among accuracy, inference latency, model size, and memory consumption under resource-constrained embedded conditions. The results are expected to provide a foundation for future real-time TinyML deployments on ESP32-S3 using live camera images.

3 SYSTEM IMPLEMENTATION AND DEVELOPMENT ENVIRONMENT

3.1 ESP32-S3 AND ESP-DL FRAMEWORK

The ESP32-S3 serves as the primary hardware platform for this evaluation, featuring an Xtensa® dual-core 32-bit LX7 microprocessor. Unlike its predecessors, the ESP32-S3 includes a specialized AI vector instruction set designed to accelerate the multiply-accumulate (MAC) operations central to deep learning (Espressif Systems, 2022).

To leverage this hardware, Espressif provides the ESP-DL library. While cross-platform frameworks such as TensorFlow Lite Micro (TFLM) prioritize portability, ESP-DL utilizes hand-optimized assembly kernels specifically for the ESP32-S3’s vector instructions. This library employs a flat-buffer-based `.espd1` model format, allowing direct mapping of weights from Flash or PSRAM into the inference engine and minimizing copying overhead (Espressif Systems, 2026b).

3.2 MODEL SELECTION AND ARCHITECTURE

This study utilizes two benchmark architectures: MobileNetV2 for image classification (Sandler et al., 2018) and YOLO11 for object detection (Jocher & Qiu, 2024). Both models are selected for their optimization toward resource-constrained environments, offering a representative view of the Pareto frontier between accuracy and computational latency.

3.3 PRE-TRAINED WEIGHTS AND BASELINE METRICS

For the empirical evaluation, we utilize official pre-trained weights for both architectures:

- **MobileNetV2:** We employ the `IMAGENET1K_V1` weights provided by Torchvision (Torchvision Maintainers, 2024), achieving a reported Top-1 accuracy of 71.88%. The model consists of approximately 3.5 million parameters and requires 300 million multiply-accumulate (MAC) operations (Sandler et al., 2018; Han et al., 2016).
- **YOLO11n:** We utilize the Nano variant weights trained on the COCO dataset. In its native FP32 format, the model achieves a mean average precision of 39.4% in terms of mAP@50–95. The architecture contains approximately 2.6 million parameters and has a computational cost of 6.5 GFLOPs (Jocher & Qiu, 2024).

These metrics serve as the baseline for evaluating the accuracy-to-performance degradation during the transition from FP32 to INT8 quantization on the target architecture.

4 METHODOLOGY AND EXPERIMENTAL SETUP

4.1 MODEL TRANSFORMATION AND QUANTIZATION PIPELINE

The transition from a high-level PyTorch environment to an embedded deployment follows a structured three-stage pipeline designed to ensure cross-framework compatibility and hardware optimization:

1. **Model Export (ONNX IR):** The pre-trained MobileNetV2 model is first exported from the PyTorch environment into the Open Neural Network Exchange (ONNX) format. This serves as a universal Intermediate Representation (IR), decoupling the model architecture from the source framework and providing a clean computational graph for downstream optimization.
2. **Post-Training Quantization (PTQ):** Utilizing the ESP-PPQ (Espressif Post-training Quantization) tool, the ONNX model undergoes INT8 linear quantization. A representative calibration dataset is used to calculate the dynamic range of activations. For the ESP32-S3, we enforce per-tensor quantization and symmetric scaling to ensure that the resulting kernels are fully compatible with the Xtensa LX7’s SIMD (Single Instruction, Multiple Data) instructions (Espressif Systems, 2026a).
3. **Code Generation and Embedding:** The optimized model is compiled into the proprietary `.espd1` format. This binary coefficient file is then integrated into the ESP-IDF (Espressif IoT Development Framework) project, where it is mapped directly into Flash or PSRAM to minimize memory-to-memory copying during inference. The comprehensive transformation workflow, from the initial PyTorch abstraction to the hardware-specific memory mapping on the ESP32-S3, is illustrated in Figure ??.

The comprehensive transformation workflow, from the initial PyTorch abstraction to the hardware-specific memory mapping on the ESP32-S3, is illustrated in Figure 1.

4.2 QUANTIZATION AND COMPRESSION STRATEGY

4.2.1 MOBILENETV2 QUANTIZATION PROFILES

Quantization maps continuous floating-point values to discrete, lower-precision representations. In this study, Post-Training Quantization (PTQ) was applied to the MobileNetV2 architecture (FP32 baseline) using four strategies to evaluate the impact on the Compression Ratio (CR) and accuracy:

1. **Default INT8:** Standard 8-bit quantization across all layers, providing a theoretical $4\times$ compression.
2. **Mixed-Precision Quantization (INT8/INT16):** This hybrid approach identifies layers with high error rates during standard 8-bit quantization analysis and manually assigns them to 16-bit precision. This strategy results in a variable compression ratio of approximately $3.8\times$, while effectively balancing model accuracy with memory footprint.
3. **Layer-wise INT8:** Optimized 8-bit quantization with independent scaling factors per layer. While the compression ratio remains $4\times$, this approach targets improved signal-to-noise ratios.

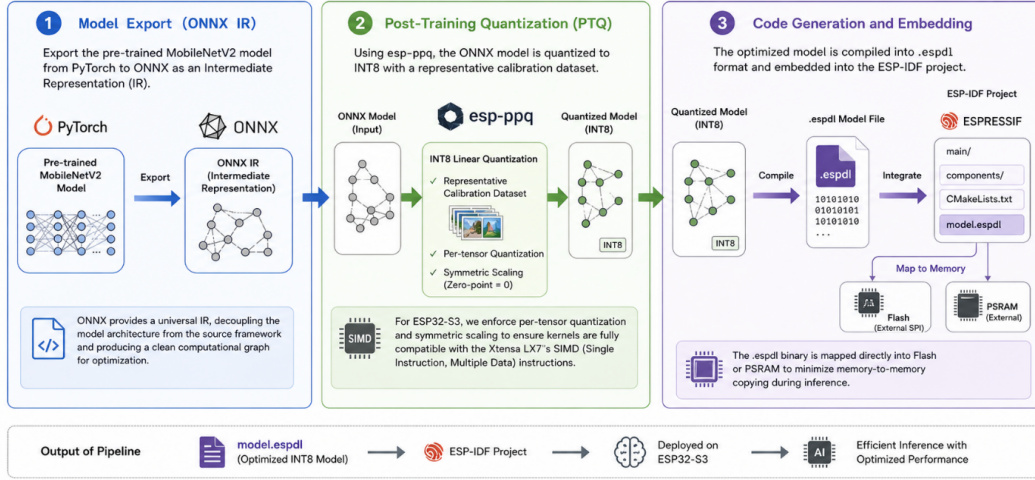


Figure 1: Workflow of the model transformation pipeline. The schematic was generated with the assistance of AI tools to visualize the ESP-DL deployment architecture.

4. **Full 16-bit (INT16):** High-precision quantization yielding a $2\times$ compression, doubling the memory load compared to 8-bit versions.

4.2.2 YOLO11N QUANTIZATION PROFILES

4.3 EVALUATION METRICS QUANTIZATION PROFILES

The models were evaluated based on three primary operational metrics:

- **Inference Latency (L):** Time required for a single forward pass in milliseconds (ms).
- **Model Confidence Score:** The softmax probability of the top-predicted class, “Pug”, on the target hardware.
- **System Stability:** Monitoring the interaction between intensive computation and the Interrupt Watchdog Timer (IWDT).

5 RESULTS AND DISCUSSION

5.1 MOBILENETV2 EVALUATION

The evaluation of MobileNetV2 on the ESP32-S3 reveals a critical trade-off between quantization bit-depth, hardware stability, and inference speed.

5.1.1 STRUCTURAL ANALYSIS AND STORAGE EFFICIENCY

As shown in Table 1, the transition from FP32 to quantized formats was essential for hardware compatibility.

- **The Flash Memory Barrier:** The baseline FP32 model, with a size of 13.66 MB, exceeds the efficient caching limits of the ESP32-S3, leading to high Bus I/O overhead.
- **Compression Linearity:** Both default 8-bit and layer-wise 8-bit quantization achieved the highest compression ratio, approximately $3.8\times$.
- **Mixed-Precision Overhead:** The mixed-precision model, with a size of 3.57 MB, shows a negligible size increase compared to pure 8-bit quantization. This confirms that manually targeting sensitive layers for 16-bit quantization provides a “sweet spot” between precision and memory footprint without significantly increasing the binary size.

Table 1: Model size and compression ratio comparison.

Quantization Type	Model Size (MB)	CR	Storage Status
FP32 (Baseline)	13.662	1.0×	Too large for cache
Full 16-bit	7.038	1.94×	High bus load
Mixed Precision	3.572	3.82×	Efficient
Default 8-bit	3.569	3.83×	Optimal footprint
Layer-wise 8-bit	3.528	3.87×	Optimal footprint

5.1.2 MOBILENETV2 OPERATIONAL PERFORMANCE AND HARDWARE STABILITY

The physical deployment results in Table 2 demonstrate that theoretical accuracy does not always translate to hardware success on the ESP32-S3. From this evaluation, we observe that:

- **The “Collapse” of 16-bit Inference:** Although the full 16-bit model theoretically maintains the highest accuracy of 71.13%, it is functionally unusable on the ESP32-S3. The extreme latency of 6150 ms exceeds the Interrupt Watchdog Timer (IWDG) threshold, causing system instability. This is because 16-bit operations cannot fully utilize the ESP32-S3’s 8-bit SIMD (Single Instruction, Multiple Data) acceleration, leading to CPU “lock-up” and a near-zero confidence score during physical testing.

Table 2: Performance evaluation matrix on ESP32-S3.

Quantization Type	Python Top-1 Acc.	S3 Latency (ms)	S3 Confidence	HW Status
Full 16-bit	71.13%	6150	< 0.001	Collapse
Mixed Precision	69.22%	1850	0.867	Success
Default 8-bit	60.32%	1680	0.931	Success
Layer-wise 8-bit	69.80%	1300	0.910	Optimal

- **The Mixed-Precision Accuracy Bridge:** The mixed-precision model successfully addressed the significant accuracy drop observed in the default 8-bit version. By keeping sensitive layers, manually identified during error analysis, at 16-bit precision, it achieved 69.22% accuracy—a nearly 9% recovery over the default 8-bit model—while maintaining a stable latency of 1850 ms. This confirms that the ESP-DL compiler can handle hybrid bit-widths to balance precision and speed.
- **Layer-wise Optimization:** The layer-wise 8-bit model emerged as the “performance leader” and the most optimal profile. It achieved the lowest latency of 1300 ms while maintaining a high accuracy of 69.80%. This suggests that providing independent scaling factors per layer allows the ESP-DL compiler to generate machine code that aligns more effectively with the Xtensa LX7 vector instructions, bypassing the efficiency losses observed in the default global scaling approach.

5.2 YOLO11N

REFERENCES

- S. Alharthi, M. Rashid, and M. Aljabri. Tinyml in industrial iot: A systematic review of applications, system components, and methodologies. *Sensors*, 26(8), 2026. doi: 10.3390/s26082550.
- F. Ben Aicha. Optimized embedded ai: Efficient implementation of cnns on esp32-cam for real-time image classification. *Computing*, 107:206, 2025. doi: 10.1007/s00607-025-01559-z.
- Y.-H. Chang, F.-C. Wu, and H.-W. Lin. Design and implementation of esp32-based edge computing for object detection. *Sensors*, 25(6), 2025. doi: 10.3390/s25061656.
- C. El Zeinaty, W. Hamidouche, G. Herrou, and D. Menard. Designing object detection models for tinyml: Foundations, comparative analysis, challenges, and emerging solutions. *ACM Computing Surveys*, 58(2), 2025. doi: 10.1145/3744339.

- Espressif Systems. *ESP32-S3 Technical Reference Manual*, 2022. Version 1.1.
- Espressif Systems. How to deploy MobileNetV2. https://docs.espressif.com/projects/espdl/en/latest/tutorials/how_to_deploy_mobilenetv2.html, 2026a.
- Espressif Systems. ESP-DL User Guide: High-performance deep learning library for Espressif SoCs. <https://github.com/espressif/esp-dl>, 2026b.
- Song Han, Huizi Mao, and William J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. In *International Conference on Learning Representations*, 2016.
- S. Heydari and Q. H. Mahmoud. Tiny machine learning and on-device inference: A survey of applications, challenges, and future directions. *Sensors*, 25(10), 2025. doi: 10.3390/s25103191.
- Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017.
- Glenn Jocher and Jing Qiu. Ultralytics YOLO11. <https://github.com/ultralytics/ultralytics>, 2024.
- Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 4510–4520, 2018.
- Torchvision Maintainers. MobileNetV2: IMAGENET1K_V1 Weights. <https://pytorch.org/vision/stable/models.html>, 2024.
- V. Tsoukas, A. Gkogkidis, E. Boumpa, and A. Kakarountas. A review on the emerging technology of tinyml. *ACM Computing Surveys*, 56(10), 2024. doi: 10.1145/3661820.
- Pete Warden and Daniel Situnayake. *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O’Reilly Media, 2019.